

American International University-Bangladesh (AIUB)

Faculty of Science and Technology

Department of Computer Science

Project Title:

Hotel Reviews Data Analysis and Rating Classification

Course: Programming in Python

Semester: Summer 25–26

Section: C

Submission Date: 09, 12, 2026 **Group-13**

Members:

Name	ID
Md. Safayat Hasan	22-46676-1
Md Toufim Islam Tasir	23-52577-2
Fahim Shahariar	24-57223-2

Submitted To:

MD. TANZEEM RAHAT

Hotel Reviews Data Analysis and Rating Classification

1. Problem Statement and Objective

Online hotel reviews carry a lot of useful signal, but on their own they don't tell you much about how a hotel stacks up against others. For this project we set out to see whether a hotel's average rating could reasonably be predicted from a handful of simple, structured features specifically its location and how many reviews it has received rather than from the text of the reviews themselves.

We aimed to:

- Get a clear picture of the original hotel review dataset before doing anything to it.
- Turn the review-level records into a cleaner, hotel-level dataset.
- Work out the number of reviews and the average rating for every hotel.
- Bucket hotels into Low, Medium, and High rating categories based on that average.
- Use latitude, longitude, and review count as the features for classification.
- Compare a simple majority-class baseline against a Decision Tree and a KNN classifier.
- Judge the models properly accuracy, weighted F1, confusion matrices, classification reports not just a single headline number.
- Be upfront about where the classification goes wrong and why.

2. Dataset, Source and Data Dictionary

We worked from a Datafiniti Hotel Reviews CSV file, `Datafiniti_Hotel_Reviews.csv`, containing 10,000 individual review records spread across 25 columns. Rather than jumping straight into modeling, we first spent time understanding what each column actually represented, then built a hotel-level dataset by grouping the review records under their respective hotel names.

Field	Description / Use
name	Hotel name; the key used to group individual reviews into hotels.
city	City where the hotel is located.
province	Province or state of the hotel.
latitude	Hotel latitude; used later as a model feature.
longitude	Hotel longitude; used later as a model feature.
categories	General category information for the hotel.
primaryCategories	The hotel's primary category.
reviews.rating	The rating given in an individual review.
review_count	How many review records exist for a given hotel.
avg_rating	The mean review rating computed for each hotel.
rating_category	The Low / Medium / High label we derive as the classification target.

3. Data Audit and Exploratory Data Analysis and Visualization Findings

The raw dataset has 10,000 rows and 25 columns. Most fields are filled in, though a handful of review-related columns have gaps here and there. Before doing anything else, we checked for duplicate rows, looked at how many unique values each column had, examined the rating distributions, and looked at where hotels were located geographically and how many reviews each one had picked up.

Once the 10,000 review records were grouped by hotel into 1,670 unique hotel names in total and merged with their review counts and average ratings, we ended up with a smaller dataset: 1,670 rows and 9 columns, with no missing values and no duplicates left after the cleaning checks.

Dataset stage	Rows	Columns
Original review-level dataset	10,000	25
Hotel-level cleaned dataset	1,670	9

Visualization Findings:

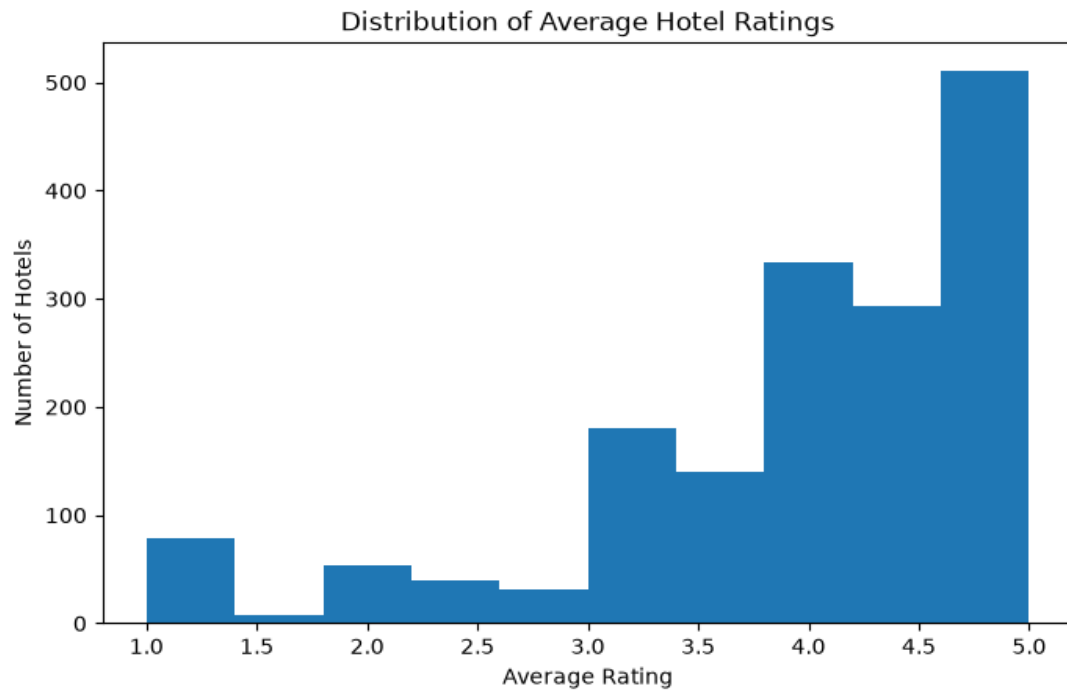


Fig: Distribution of Average Hotel Ratings

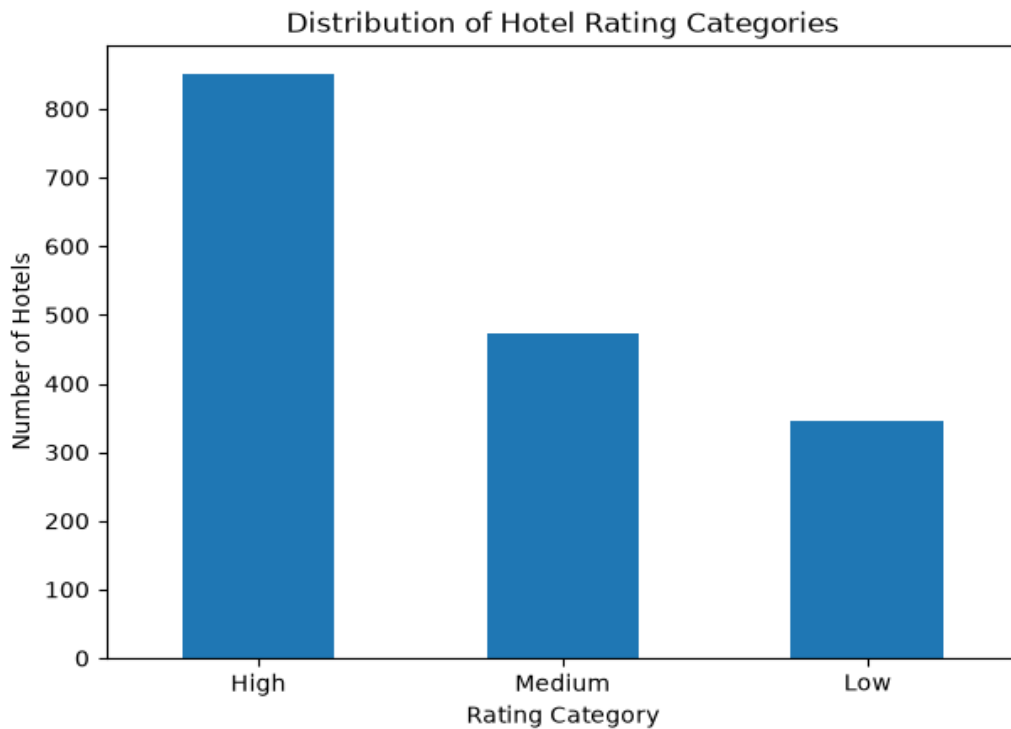


Fig: Rating category distribution (Low / Medium / High)

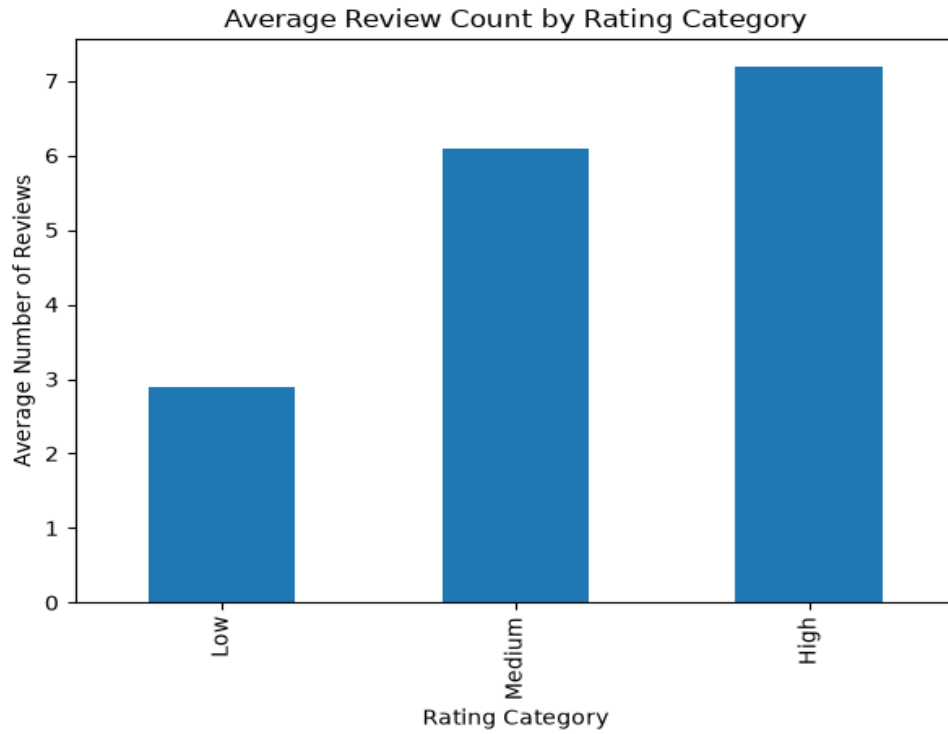


Fig: Review count analysis

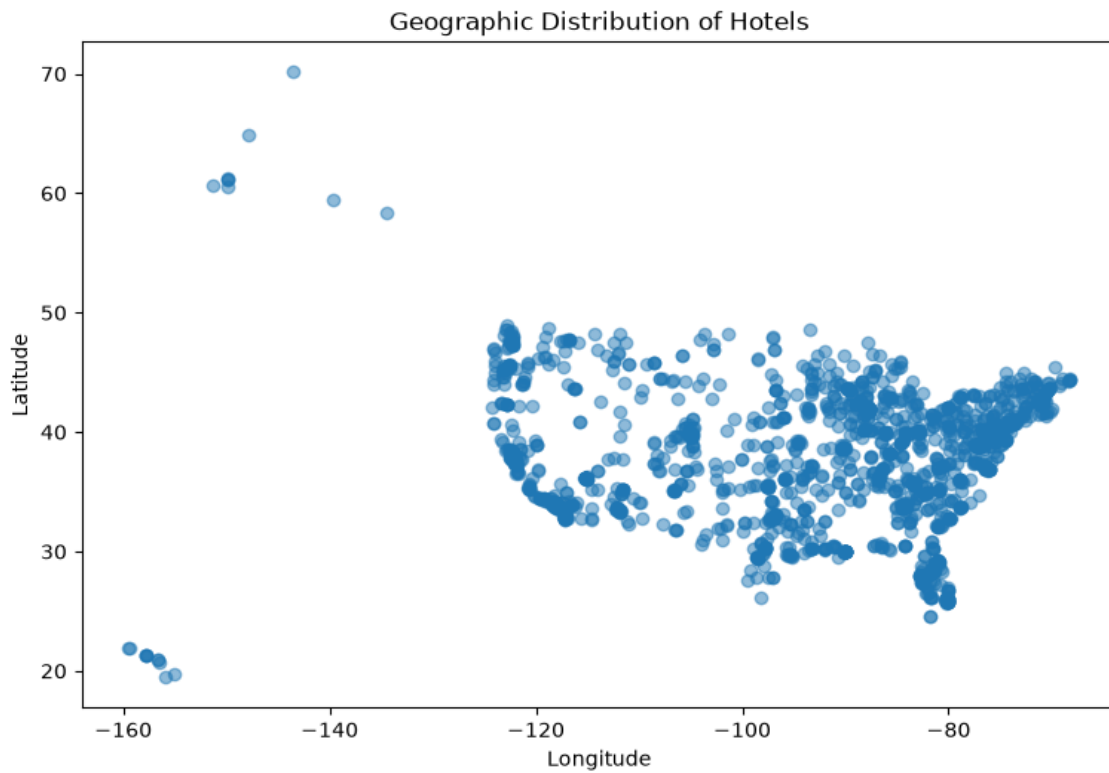


Fig: Geographic/location visualization

4. Split strategy, preprocessing decisions, and leakage controls

A number of columns weren't going to be useful for the modeling stage, so we dropped them things like identifiers, source URLs, the review text and title, and various pieces of reviewer metadata. We converted the review date into a proper datetime type, then aggregated everything by hotel name: `review_count` came from simply counting the review records per hotel, and `avg_rating` from averaging `reviews.rating` within each group.

The target variable, `rating_category`, was created from `avg_rating` using three bins:

Average rating	Category
0 to 3	Low
Above 3 to 4	Medium
Above 4 to 5	High

That gave us the following class distribution:

Category	Hotels	Percentage
Low	346	20.72%
Medium	474	28.38%
High	850	50.90%

For features, we kept it simple and used latitude, longitude, and `review_count` three numerical columns. Since KNN relies on distance calculations, we scaled the features with a `StandardScaler`, fitting it on the training data only and then applying that same transformation to the test set.

5. Baseline definition and result

Before trusting any classifier, it's worth knowing what a naive strategy would achieve. Our baseline simply predicts the most common class every time in this case, High and it reaches 50.90% accuracy.

6. Candidate Models

6.1 K-Nearest Neighbours (KNN)

We trained KNN on the scaled latitude, longitude, and review_count features, trying several values of $k=3, 5, 7, 9$, and 11 before settling on $k=11$, which gave the best accuracy among those tested. The final KNN result reported in the model performance summary was 43.41% accuracy with a weighted F1 score of 0.390.

6.2 Decision Tree

We also tested a Decision Tree on the same three features, sweeping through max_depth values of 2, 3, 4, 5, 6, 7, 8, and 10. A depth of 3 produced the best accuracy, so that's the configuration we kept. That final tree reached 51.20% accuracy, with a weighted F1 score of roughly 0.350.

7. Final Model Evaluation and Metrics

Model	Accuracy	Weighted F1
Majority Baseline	50.90%	0.343
Decision Tree	51.20%	0.350
KNN	43.41%	0.390

The Decision Tree edges out the baseline on raw accuracy, but only by about 0.30 percentage points not a convincing margin. Interestingly, KNN comes in lower on accuracy yet posts the best weighted F1 of the three. Looking closer at the Decision Tree's classification report, the High class has near-perfect recall (1.00), while Low and Medium are barely being picked up at all.

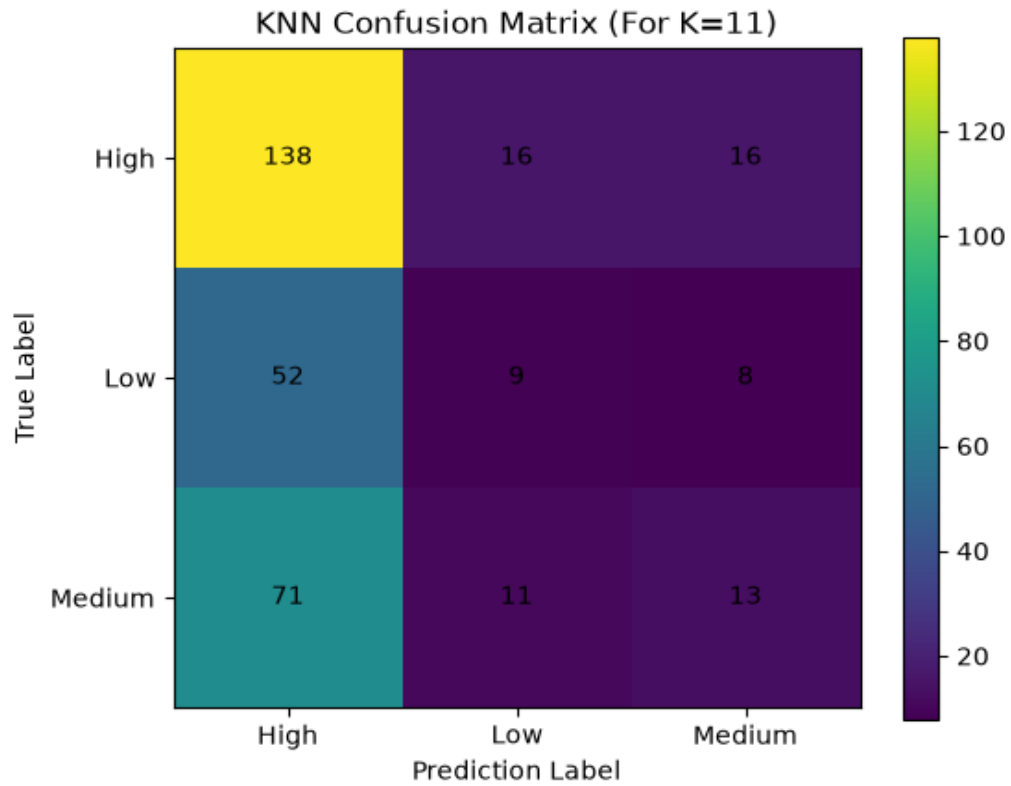


Fig: KNN Confusion Matrix Visual

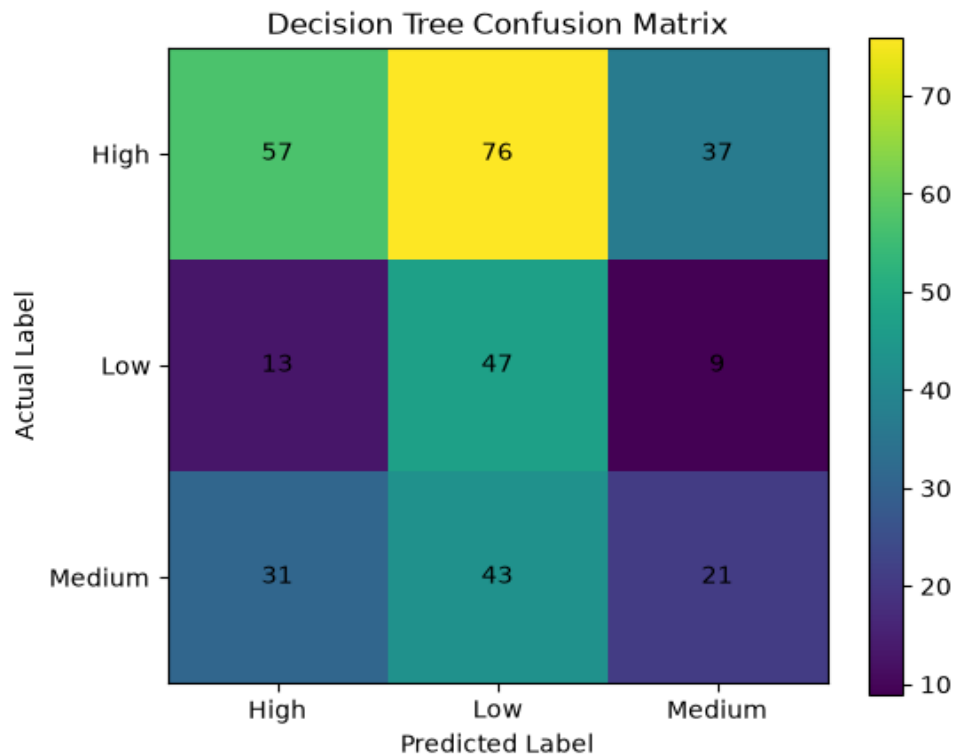


Fig: Decision Tree Confusion Matrix Visual

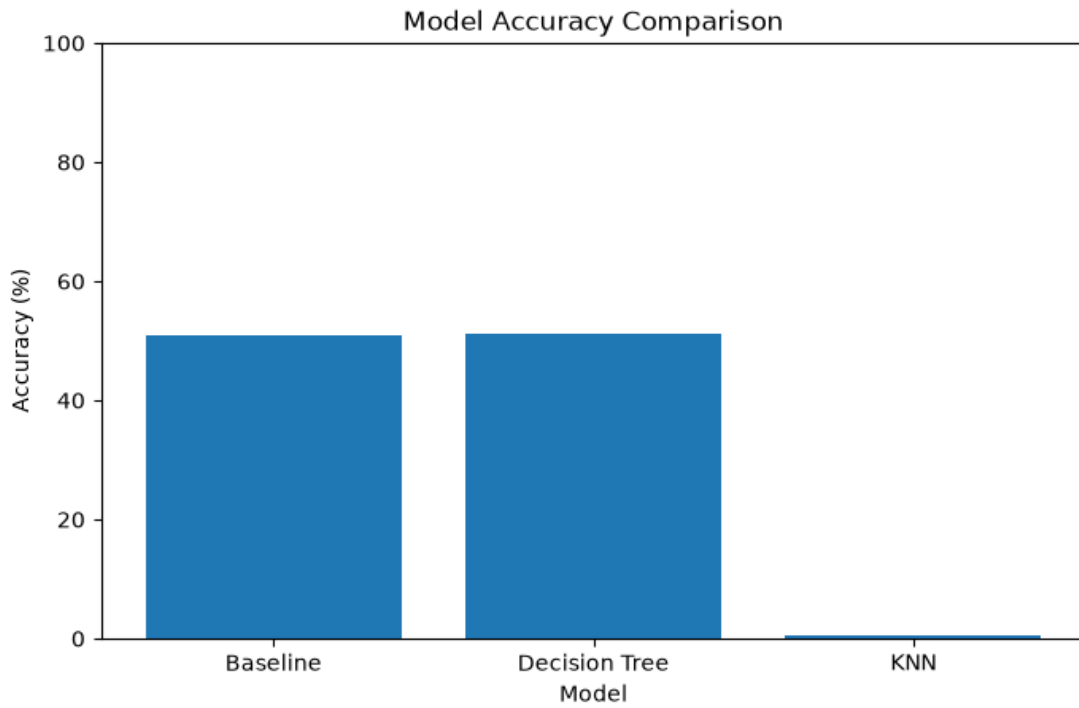


Fig: Model performance comparison

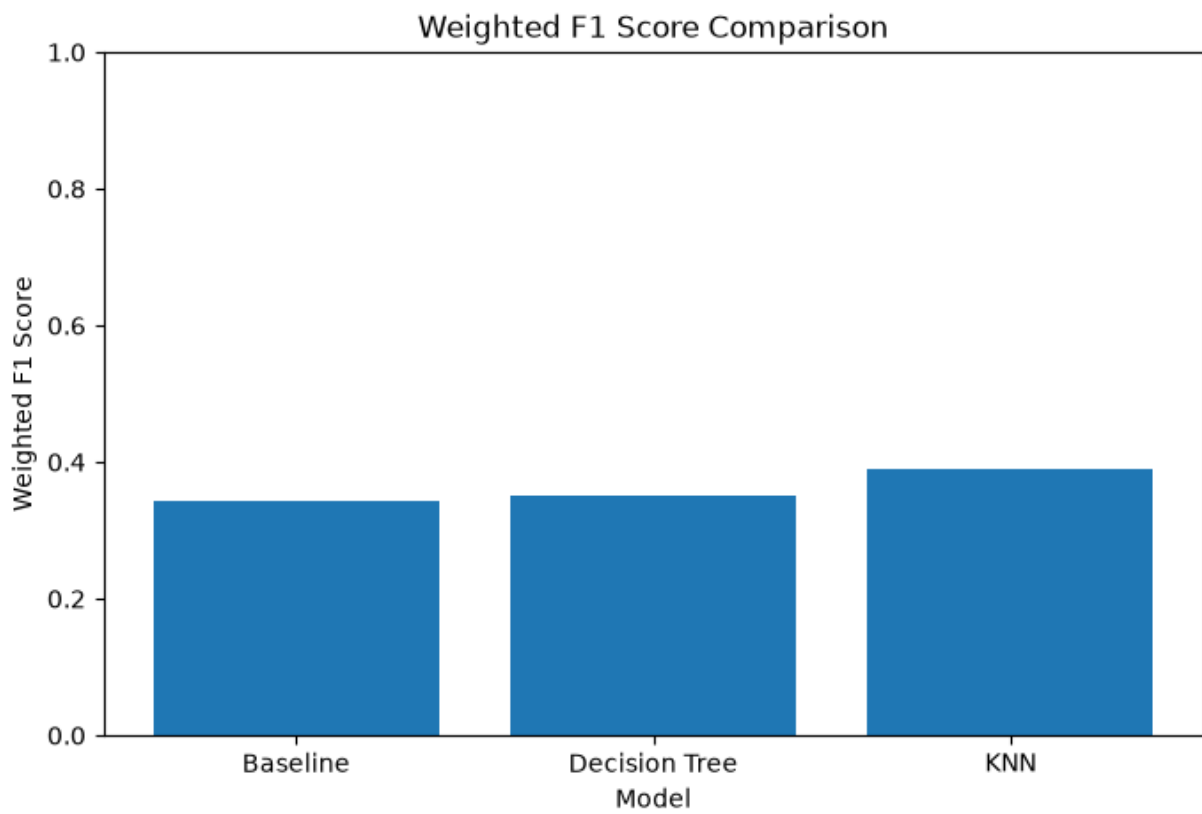


Fig: Final model comparison

8. Confusion Matrix and Error Analysis

Actual \ Predicted	High	Low	Medium
High	170	0	0
Low	68	1	0
Medium	95	0	0

This table tells the real story behind the accuracy figures. The Decision Tree predicts High for almost every test sample 170 of those are correctly High, but 68 out of 69 Low hotels and all 95 Medium hotels get mislabeled as High. Out of 334 test samples in total, only 171 predictions were correct, leaving 163 wrong. That's essentially the model defaulting to the majority class, which also explains why its accuracy sits so close to the baseline despite technically being the 'best' model on that metric.

9. Model Limitations

-
- The target classes are imbalanced High makes up roughly 50.90% of all hotels, which skews both training and evaluation.
 - We relied on just three numerical features: latitude, longitude, and review_count.
 - Those features don't seem to carry enough signal to cleanly separate Low, Medium, and High ratings.
 - The Decision Tree in particular leans heavily toward predicting High.
 - Accuracy by itself hides how unevenly the models perform across classes.
 - KNN struggled with recall for Low and Medium in much the same way.
 - Many features likely needed to meaningfully improve classification.

10. Reproducibility information

The project lives in a Jupyter notebook and runs inside a dedicated Python virtual environment. A fixed random_state=42 is used throughout for the train/test split and wherever else randomness comes into play, so results should be reproducible. Package versions are pinned separately in requirements.txt. To rerun the project end to end: place the dataset in the expected data directory, install the dependencies from requirements.txt, and execute the notebook cells in order from start to finish.

11. Conclusion and Future Improvements

Overall, this project walked through a complete, if modest, hotel review analysis and classification pipeline starting from raw review-level data and ending with a working, if imperfect, classifier. We turned the original data into a hotel-level dataset with review counts and average ratings, then built rating categories to use as a classification target.

Comparing a majority-class baseline, KNN, and a Decision Tree, the Decision Tree came out ahead on accuracy at 51.20%, while KNN posted the better weighted F1 at 0.390. But the confusion matrix makes clear that the Decision Tree's edge comes largely from defaulting to the High category, which leaves it performing poorly on Low and Medium hotels.

Given that, these models are best thought of as a starting point rather than something ready to reliably predict hotel ratings on its own. Future work could bring in additional features text-based signals from the reviews themselves, for instance, or explore different preprocessing and modeling strategies to get better separation between the rating categories.

12. References and Contributions

[1] Datafiniti, "Datafiniti Hotel Reviews," Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/datafiniti/hotel-reviews>.

Software and libraries used: Python, NumPy, Pandas, Matplotlib, and Scikit-learn.

Group Member Contributions:

Name: MD TOUFIM ISLAM TASIR

ID: 23-52577-2

Main project development, including data loading, data cleaning and preprocessing, initial dataset audit, feature preparation, train-test splitting, KNN development, and other core technical/model-development tasks.

Name: MD. SAFAYAT HASAN KHAN HREDOY

ID: 22-46676-1

Hotel-level EDA, Decision Tree development, model evaluation, visualization, model comparison, error analysis, final model selection, conclusion, and final code integration and testing.

Name: FAHIM SHAHARIAR

ID: 24-57223-2

Dataset documentation, data dictionary, README, references, report organization and formatting, and presentation preparation.